



The Salivo Programming Language

Master Architecture Manual, Compiler Subsystems & Runtime Specification

Language Version 1.0 · Fixed-Point Self-Hosting Stage 35.4 · Co-Founder Master Edition

BUILD TIMESTAMP: SEPTEMBER 12, 2026 · ANTIGRAVITY IDE ENGINE · ZERO-UNDERSCORE LAW

01

Type Integrity

Clean PascalCase types with the zero-underscore standard, coupled with flexible naming for functions, variables, and struct fields.

02

Stream Architecture

Standardized `><` stream imports providing expressive, visual dependency declarations and zero syntax fragmentation.

03

Deterministic RAI

First-class `pub trait Drop` with automated ARC memory management and deterministic destruction upon scope exit.

04

Multi-Stage Bootstrap

Authentic mathematical fixed-point self-hosting verification: `SHA256(C2) == SHA256(C3)` with zero synthetic copying.

Table of Contents

Systematic roadmap of core language syntax, type semantics, runtime engine, and standard library modules.

PART 1 Part I: Language Fundamentals & Core Syntax 1.1 Lexical Structure & Clean Language Design 1.2 Modules & Stream Import Directives 1.3 Variables, Mutability, & Scope 1.4 Data Types & Literals 1.5 Operators & Expressions 1.6 Control Flow 1.7 Functions & Procedures	PART 5 Part V: Complete Standard Library Reference 5.1 <code>salivo.std.prelude</code> & <code>salivo.std.core</code> 5.2 <code>salivo.std.strings</code> 5.3 <code>salivo.std.collections</code> 5.4 <code>salivo.std.math</code> 5.5 <code>salivo.std.io</code> 5.6 <code>salivo.std.fs</code> 5.7 <code>salivo.std.crypto</code> 5.8 <code>salivo.std.sync</code> 5.9 <code>salivo.std.time</code> 5.10 <code>salivo.std.net</code> 5.11 <code>salivo.std.task</code> 5.12 Auxiliary Modules
PART 2 Part II: Type System, Structs, Enums & Traits 2.1 Structs & Generic Types 2.2 Enums & Algebraic Data Types 2.3 Pattern Matching (<code>match</code>) 2.4 Traits & Method Implementations	
PART 3 Part III: Postfix Modifiers & String Interpolation 3.1 Postfix Modifiers Reference 3.2 Modifier Chaining 3.3 Formatted String Interpolation (<code>\$"..."</code>) 3.4 String Literal Auto-Coercion 3.5 The <code>format!</code> Macro & Trait Formatting	
PART 4 Part IV: Low-Level Memory Engine & Smart Pointers 4.1 Heap Allocation Engine 4.2 Raw Memory Operations & Intrinsics 4.3 Slices & Spans 4.4 Smart Pointers: <code>Box<T></code>, <code>Arc<T></code>, <code>Weak<T></code>, Arena	PART 6 Part VI: Tooling, Compiler CLI & Project Ecosystem 6.1 Compiler Commands (<code>sf</code>) 6.2 Compiler Optimization Pipeline & Performance Tuning 6.3 Package Manager (<code>spm</code>) 6.4 Code Quality & Tooling
	PART 7 Part VII: Complete Working Examples Example 1: High-Performance Data Pipeline with Collections & Hashing Example 2: Thread-Safe State & Atomic Concurrency Example 3: File System Inspector & Path Manipulation

Quick Reference & Syntax Cheat Sheet

Feature	Salivo Syntax	Notes
Module Header	<code>module mypackage.core;</code>	Required at the top of every file
Stream Imports	<code>><salivo.std.io;</code> or <code>><salivo.std.math;</code>	Grouped: <code>><salivo.std.core::{assert, assertEq};</code>
Immutable Variable	<code>let name = "Salivo";</code>	Bound value cannot be reassigned
Mutable Variable	<code>let mut count = 0;</code> or <code>mut count = 0;</code>	Reassignable with <code>=</code> or <code>+=</code>
Constant	<code>const MAX_SIZE = 4096;</code>	Evaluated at compile time
Global Binding	<code>global let PORT = 8080;</code>	Package-scoped global
Function	<code>pub func add(a: int, b: int) -> int { return a + b; }</code>	Returns value with <code>-> Type</code>
Entry Point	<code>func main() -> int { outln("Hello"); return 0; }</code>	Returns exit status code
Conditionals	<code>if (x > 0) { ... } else if (x == 0) { ... } else { ... }</code>	Parentheses around conditions
While Loop	<code>while (count < 10) { count = count + 1; }</code>	Standard while loop
For Loop	<code>for (let mut i = 0; i < n; i = i + 1) { ... }</code>	3-clause C-style loop
Struct	<code>struct Point { x: int, y: int }</code>	Generic: <code>struct Pair<T, U> { first: T, second: U }</code>
Enum	<code>enum Status { Pending, Active, Done }</code>	Algebraic: <code>enum Option<T> { Some(T), None }</code>
Pattern Match	<code>match val { 0 => "zero", 1 => "one", _ => "other" }</code>	Strictly exhaustive
Try Operator	<code>let data = open("file.txt"?);</code>	Unwraps <code>Result</code> / <code>Option</code> or early-returns
Trait	<code>trait Show { func show(self) -> string; }</code>	Interface contracts
Implementation	<code>impl Show for Point { func show(self) -> string { ... } }</code>	Trait and struct methods
Postfix Modifiers	<code>val:trim:upper</code> , <code>num:str</code> , <code>text:slice(0, 5)</code> , <code>text:len</code>	Left-to-right chaining
String Coercion	<code>"literal"</code> seamlessly coerces to <code>StrSlice</code> / <code>String</code>	Zero boilerplate string passing
String Interpolation	<code> \$"User: {name:trim:upper}, ID: {id:str}, Score: {score}"</code>	Formatted templates
Standard Input	<code>let line = in();</code> or <code>let num = in():int;</code>	Flushes stdout automatically
Standard Output	<code>out("Prompt: ");</code> and <code>outln("Message");</code>	Prints to stdout

Part I: Language Fundamentals & Core Syntax

1.1 Lexical Structure & Clean Language Design

Salivo enforces a clean, modern type system while providing maximum flexibility for functions, variables, and system programming:

- **Data Types (Zero-Underscore Standard):** Structs, enums, traits, and type annotations strictly use clean PascalCase without underscores (`Point`, `Option`, `Result`, `UserProfile`). The underscore `_` is strictly never a datatype.
- **Functions, Variables, & Struct Fields (Flexible Naming):** Functions, method names, variables, and struct fields freely support both modern `camelCase` and `snake_case` with underscores (`pathJoin` or `path_join`, `assertEq` or `assert_eq`, `userId` or `user_id`, `maxLineWidth` or `max_line_width`). Both forms are fully first-class and interoperable across the standard library, tooling, and user code.
- **Numbers & Wildcards:** Numbers support underscore digit separators for readability (e.g., `1_000_000`), and `_` is used for pattern match wildcards and discarded bindings (`let _ = expr;`).

```
SALIVO

// ✅ Clean Data Types (Zero-Underscores)
struct UserSession {
    int userId;
    string token;
    bool isActive;
}

// ✅ Flexible Function & Variable Identifiers
pub func get_user_session(int user_id) -> UserSession {
    let timeout_ms = 30_000;
    // ...
}
```

Comments

```
SALIVO

// Single-line comment

/*
  Multi-line block comment
*/

/// Documentation docstring attached to following declaration
pub func calculate() -> int { return 42; }
```

1.2 Modules & Stream Import Directives

Every Salivo file begins with an explicit module namespace declaration. Modules are organized in dot-separated hierarchies.

Module Declaration

```
SALIVO  
  
module myapp.network.client;
```

Stream Import Syntax (<>)

Salivo uses the **Stream Header Import Operator** (<>) exclusively for importing modules, types, and functions. (The legacy `import` keyword is deprecated).

```
SALIVO  
  
// 1. Full module stream import  
><salivo.std.io;  
><salivo.std.math;  
  
// 2. Selective symbol import with destructuring  
><salivo.std.core::{assert, assertEq, Option, some, none};  
  
// 3. Selective cryptographic and filesystem primitives  
><salivo.std.crypto::{sha256Hex, secSeed};  
><salivo.std.fs::{pathJoin, pathExists, open, close};  
  
// 4. Custom package imports  
><mypackage.services.auth::{authenticate, Session};
```

1.3 Variables, Mutability, & Scope

Variables in Salivo are immutable by default when declared with `let`.

 SALIVO

```
// Immutable binding (cannot be reassigned)
let max_connections = 100;
let host = "127.0.0.1";

// Mutable binding
let mut counter = 0;
counter = counter + 1;
counter += 5;

// Explicit type annotations
int port = 8080;
float ratio = 1.618;
bool is_running = true;
string application_name = "SalivoEngine";

// Compile-time constants
const BUFFER_CAPACITY = 8192;
const MAX_RETRIES = 3;

// Global package-scoped bindings
global let SYSTEM_VERSION = "1.0.0";
global int ACTIVE_WORKERS = 0;
```

1.4 Data Types & Literals

Primitive Data Types

- `int`: 64-bit signed two's complement integer (`-9223372036854775808` to `9223372036854775807`).
- `float`: 32-bit IEEE 754 single-precision floating point.
- `double`: 64-bit IEEE 754 double-precision floating point.
- `bool`: Boolean values `true` or `false` .
- `char`: Unicode scalar codepoint value (represented internally as UTF-32 scalar).
- `string`: UTF-8 immutable reference-counted string sequence.
- `byte`: 8-bit unsigned octet (0 to 255).
- `void`: Unit type representing the absence of a value.

Literals

 SALIVO

```
let dec = 42;
let hex = 0x2A;
let bin = 0b101010;
let flt = 3.14159;
let flag = true;
let ch = 'Z';
let str_val = "Salivo Language\n";
let raw_escaped = "C:\\Windows\\System32";
```

1.5 Operators & Expressions

 SALIVO

```
// Arithmetic
let sum = 10 + 20;
let diff = 30 - 15;
let prod = 4 * 5;
let quot = 100 / 4;
let rem = 29 % 5;

// Comparison
let eq = (sum == 30);
let ne = (diff != 0);
let lt = (prod < 50);
let lte = (prod <= 20);
let gt = (quot > 10);
let gte = (quot >= 25);

// Logical
let both = (eq && !lt);
let either = (eq || lt);

// Bitwise
let bit_and = 0xFF & 0x0F;
let bit_or = 0xF0 | 0x0F;
let bit_xor = 0xAA ^ 0x55;
let bit_shl = 1 << 4;
let bit_shr = 16 >> 2;

// Assignment Compound Operators
let mut num = 10;
num += 5;
num -= 2;
num *= 3;
num /= 2;

// Postfix Try Operator (?) for Error Propagation
// Unwraps Ok(v)/Some(v) or early-returns Err(e)/None from the current function
let file_handle = open("config.json"?);
let user_id = find_user("admin"?);
```

1.6 Control Flow

If / Else Conditions

 SALIVO

```
if (score >= 90) {
    grade = "A";
} else if (score >= 80) {
    grade = "B";
} else if (score >= 70) {
    grade = "C";
} else {
    grade = "F";
}
```

While Loop

```
SALIVO

let mut i = 0;
while (i < 10) {
    if (i == 5) {
        i = i + 1;
        continue;
    }
    println($"Step: {i}");
    i = i + 1;
}
```

3-Clause For Loop

```
SALIVO

for (let mut idx = 0; idx < 10; idx = idx + 1) {
    if (idx == 8) {
        break;
    }
    println($"Index: {idx}");
}
```

1.7 Functions & Procedures

Functions are declared with `func`. The return type is defined after `->`. If omitted, the return type defaults to `void`.

```
SALIVO

// Basic function
func calculate_area(width: int, height: int) -> int {
    return width * height;
}

// Public generic function
pub func identity<T>(item: T) -> T {
    return item;
}

// Function with no return value
pub func log_message(msg: string) {
    println($"[LOG] {msg}");
}

// Program Entry Point
func main() -> int {
    let area = calculate_area(10, 20);
    println($"Area: {area}");
    return 0;
}
```

Part II: Type System, Structs, Enums & Traits

2.1 Structs & Generic Types

Structs define custom composite data structures.

```
pub struct Vector2 {
    x: float,
    y: float,
}

// Generic Struct
pub struct KeyValuePair<K, V> {
    key: K,
    value: V,
    is_valid: bool,
}

// Instantiation & Field Access
let v = Vector2 { x: 3.5, y: 7.2 };
println($"Vector: ({v.x}, {v.y})");

let pair = KeyValuePair<string, int> {
    key: "timeout_ms",
    value: 5000,
    is_valid: true,
};
```

2.2 Enums & Algebraic Data Types

Enums can be standard tagged discriminants or algebraic sum types with attached payloads.

```
// Tagged Enum
pub enum HttpMethod {
    Get,
    Post,
    Put,
    Delete,
    Patch,
}

// Algebraic Tagged Union with Payloads
pub enum WebResponse {
    Success(string),
    Redirect(string, int),
    ClientError(int, string),
    ServerError(string),
}
```

2.3 Pattern Matching (match)

Pattern matching is exhaustive and supports destructuring enum variants and constants.

```
pub func handle_response(resp: WebResponse) -> string {
    return match resp {
        WebResponse::Success(body) => $"200 OK: {body}",
        WebResponse::Redirect(url, code) => $"Redirect {code} to {url}",
        WebResponse::ClientError(status, err) => $"Error {status}: {err}",
        WebResponse::ServerError(msg) => $"500 Internal Error: {msg}",
        _ => "Unknown status",
    };
}

let status = HttpMethod::Get;
match status {
    HttpMethod::Get => outln("Performing GET request"),
    HttpMethod::Post => outln("Performing POST request"),
    _ => outln("Other HTTP method"),
}
```

2.4 Traits & Method Implementations

Traits define contracts for polymorphism. Methods and traits are attached to structs using `impl`.

```
pub trait Printable {
    func toString(self) -> string;
}

pub struct User {
    id: int,
    username: string,
}

// Implementing a Trait
impl Printable for User {
    func toString(self) -> string {
        return $"User(id={self.id}, username={self.username})";
    }
}

// Adding Methods Directly to a Struct
impl User {
    func new(id: int, name: string) -> User {
        return User { id: id, username: name };
    }

    func isadmin(self) -> bool {
        return self.id == 1;
    }
}

// Usage
let u = User::new(1, "guest");
outln(u.toString());
if (u.isadmin()) {
    outln("Administrator access granted.");
}

// Automatic Scope Cleanup Protocol (Drop Trait)
pub trait Drop {
    func drop(self);
}

impl Drop for User {
    func drop(self) {
        outln($"Cleaning up resources for user {self.username}...");
    }
}

// Display and Debug Traits for Formatting
pub trait Display {
    func display(self) -> String;
}

pub trait Debug {
    func debug(self) -> String;
}

impl Display for User {
    func display(self) -> String {
        return $"User({self.username} #{self.id})";
    }
}

impl Debug for User {
    func debug(self) -> String {
        return $"User {{ id: {self.id}, username: \"{self.username}\" }}";
    }
}
```

} }

Part III: Postfix Modifiers & String Interpolation

Salivo features a unique **Postfix Modifier Engine** that eliminates nested wrapper functions in favor of concise, left-to-right chaining.

3.1 Postfix Modifiers Reference

Modifier	Description	Example	Result
<code>:int</code>	Converts string or float to integer	<code>"42":int</code>	<code>42</code>
<code>:float</code>	Converts string or integer to float	<code>"3.14":float</code>	<code>3.14</code>
<code>:str</code>	Converts integer, float, or object to string	<code>100:str</code>	<code>"100"</code>
<code>:trim</code>	Trims leading and trailing whitespace	<code>" salivo ":trim</code>	<code>"salivo"</code>
<code>:upper</code>	Converts string to uppercase	<code>"hello":upper</code>	<code>"HELLO"</code>
<code>:lower</code>	Converts string to lowercase	<code>"WORLD":lower</code>	<code>"world"</code>
<code>:len</code>	Returns byte length of string or collection	<code>"data":len</code>	<code>4</code>
<code>:slice(a, b)</code>	Extracts substring from index <code>a</code> to <code>b</code>	<code>"salivolang":slice(0, 6)</code>	<code>"salivo"</code>

3.2 Modifier Chaining

Modifiers chain naturally from left to right:

SALIVO

```
let input_text = "  HELLO SALIVO DEVELOPERS  ";
let cleaned = input_text:trim:lower:slice(0, 12);
// Result: "hello salivo"
```

3.3 Formatted String Interpolation (`"${...}"`)

Templates embed variables, expressions, and modifier chains directly:

 SALIVO

```
let user = "  developer  ";
let score = 98.6;
let rank = 1;

let summary = $"Candidate: {user:trim:upper}, Rank: #{rank:str}, Score: {score:float}";
outln(summary);
// Output: Candidate: DEVELOPER, Rank: #1, Score: 98.6
```

3.4 String Literal Auto-Coercion

Salivo features automatic string literal coercion. Whenever a function, struct field, or variable declaration expects a borrowed `StrSlice` or owned `String`, a raw `"string literal"` is seamlessly coerced at compile time without requiring manual conversion boilerplate:

 SALIVO

```
><salivo.std.fs::{pathJoin};
><salivo.std.strings::{pushStr, stringNew};

// String literals automatically coerce to StrSlice / String parameters!
let joined = pathJoin("var", "log/salivo.log");

let mut s = stringNew();
s = pushStr(s, "Automated Coercion");
```

3.5 The `format!` Macro & Trait Formatting

Salivo provides a high-performance, compile-time validated `format!` macro that expands expressions into owned `String` instances with zero unnecessary allocations.

Placeholders & Dispatch Modes

- `{}` — **Display formatting**: Invokes the `Display` trait (`func display(self) -> String`) on custom types, or native string formatting on primitives (`int`, `float`, `bool`, `string`, `StrSlice`, `String`).
- `{:?}` — **Debug formatting**: Invokes the `Debug` trait (`func debug(self) -> String`) on custom types, or escaped quote debugging on strings.

```
><salivo.std.core::{Display, Debug, outln};
><salivo.std.strings::{String};

pub struct Point {
    int x;
    int y;
}

impl Display for Point {
    func display(self) -> String {
        return $"Point({self.x}, {self.y})";
    }
}

pub func main() -> int {
    let p = Point { x: 10, y: 25 };
    let text = format!("Coordinate: {}, Label: {:?}" , p, "target_alpha");
    outln(text);
    // Output: Coordinate: Point(10, 25), Label: "target_alpha"
    return 0;
}
```

Safety & Memory Invariants

1. **Checked Allocation Arithmetic:** Aggregate segment lengths and final buffer allocations are evaluated through runtime-checked arithmetic (`salivo_checked_add_len`), preventing 64-bit integer wrapping vulnerabilities and panicking cleanly if sizes exceed memory limits.
2. **Deterministic Temporary Deallocation:** Temporary heap allocations produced dynamically during formatting (such as custom `Display` returned `String` buffers or escaped debug C buffers) are tracked and freed exactly once upon formatting completion.
3. **Authoritative Length with Embedded NUL:** Formatting preserves exact byte counts and embedded NUL (`\0`) bytes without `strlen` truncation.

Part IV: Low-Level Memory Engine & Smart Pointers

The `salivo.std.mem` module gives full control over heap memory, reference counting, and raw layout intrinsics.

4.1 Heap Allocation Engine

```

SALIVO

><salivo.std.mem;

// Allocate 64 bytes with 8-byte alignment
let addr = alloc(64, 8);

// Allocate zero-initialized memory
let zero_addr = alloczero(128, 8);

// Check if pointer is currently live
let is_live = isallocated(addr);

// Reallocate buffer to larger size (from 64 to 256 bytes)
let new_addr = realloc(addr, 64, 256, 8);

// Deallocate when finished
dealloc(new_addr, 256, 8);
dealloc(zero_addr, 128, 8);
```

4.2 Raw Memory Operations & Intrinsics

 SALIVO

```
><salivo.std.mem;

let buffer = alloc(32, 8);

// Write and read 64-bit integers at offsets
writei64(buffer, 0, 1000);
writei64(buffer, 8, 2000);
let val0 = readi64(buffer, 0); // 1000
let val1 = readi64(buffer, 8); // 2000

// Write and read single bytes
writebyte(buffer, 16, 255);
let b = readbyte(buffer, 16); // 255

// Zero out a region of memory
zero(buffer, 0, 32);

// Copy memory from source to destination
let dest = alloc(32, 8);
copy(buffer, dest, 32);

// Pointer arithmetic and string conversions
let s_ptr = strtoupper("Hello Memory");
let str_back = ptrtostr(s_ptr);

dealloc(buffer, 32, 8);
dealloc(dest, 32, 8);
```

4.3 Slices & Spans

Non-owning views over contiguous memory buffers:

 SALIVO

```
><salivo.std.mem;

let raw = alloc(80, 8);
for (let mut i = 0; i < 10; i = i + 1) {
    writei64(raw, i * 8, (i + 1) * 10);
}

// Create slice view
let s: Slice<int> = fromrawparts<int>(raw, 10);
let length = slicelen<int>(s); // 10
let el0 = sliceget<int>(s, 0); // 10
let el4 = sliceget<int>(s, 4); // 50

// Modify through slice view
sliceset<int>(s, 0, 999);

// Create subslice (indices 2 to 6)
let sub = slicesub<int>(s, 2, 6);

dealloc(raw, 80, 8);
```

4.4 Smart Pointers: Box<T>, Arc<T>, Weak<T>, Arena

```
><salivo.std.mem;

// 1. Box<T> (Heap allocated single ownership)
let b = boxnew<int>(42, 8);
let value = deref<int>(b); // 42

// 2. Arc<T> (Atomic Reference Counting)
let arc_node = arcnew<string>("Shared State", 16);
let cloned_arc = retain<string>(arc_node);
let strong_cnt = strongcount<string>(cloned_arc); // 2

let released = release<string>(cloned_arc);
let final_release = release<string>(arc_node); // Automatically deallocated

// 3. Arena Allocator
let a = arena(1048576); // 1 MB Arena
```

Part V: Complete Standard Library Reference

Every module in the Salivo Standard Library (`salivo.std.*`) is described below with complete function signatures, descriptions, and code examples.

5.1 `salivo.std.prelude` & `salivo.std.core`

Automatically imported into every Salivo compilation unit.

Functions, Structs & Traits

`Option<T>`

```
pub struct Option<T> {  
  value: T,  
  is_some: bool,  
}
```

- `some<T>(val: T) -> Option<T>`: Constructs an `Option` containing a value.
- `none<T>() -> Option<T>`: Constructs an empty `Option`.

`Result<T, E>`

```
pub struct Result<T, E> {  
  value: T,  
  error: E,  
  is_ok: bool,  
}
```

- `ok<T, E>(val: T) -> Result<T, E>`: Constructs a successful `Result`.
- `err<T, E>(e: E) -> Result<T, E>`: Constructs a failed `Result`.

Diagnostics & Assertions

- `panic(msg: string)`: Aborts execution with a diagnostic trace.
- `assert(cond: bool, msg: string)`: Panics if `cond` is false.

- `assertEq(a: int, b: int, msg: string) / asserteq(...)` : Panics if `a != b`.
- `assertStrEq(a: string, b: string, msg: string) / assertstreq(...)` : Panics if `a != b`.
- `assertNe(a: int, b: int, msg: string) / assertne(...)` : Panics if `a == b`.
- `assertStrNe(a: string, b: string, msg: string) / assertstrne(...)` : Panics if `a == b`.

Standard Traits

- `trait Display { func display(self) -> string; }`
- `trait Debug { func debug(self) -> string; }`
- `trait Iterator<T> { func next(self) -> Option<T>; }`
- `trait Drop { func drop(self); }` : Standard automated RALL cleanup protocol.

Core Conversions & Formatting

- `str(v: int) -> string` : Converts integer to string.
- `intstr(v: int) -> string` : Converts integer to string.
- `strint(s: string) -> int` : Parses string to integer.
- `floatstr(f: float) -> string` : Converts float to string.
- `strfloat(s: string) -> float` : Parses string to float.
- `len(s: string) -> int` : Returns string length.
- `trim(s: string) -> string` : Strips whitespace from both ends.
- `upper(s: string) -> string` : Converts string to uppercase.
- `lower(s: string) -> string` : Converts string to lowercase.
- `slice(s: string, start: int, end: int) -> string` : Substring from `start` to `end`.
- `charat(s: string, idx: int) -> int` : Returns Unicode codepoint at index `idx`.
- `find(s: string, sub: string) -> int` : Returns first index of `sub`, or `-1`.
- `contains(s: string, sub: string) -> bool` : Returns `true` if `sub` exists in `s`.
- `startswith(s: string, prefix: string) -> bool / startswith(...)` : Returns `true` if `s` begins with `prefix`.
- `endswith(s: string, suffix: string) -> bool / endswith(...)` : Returns `true` if `s` terminates with `suffix`.
- `replace(s: string, target: string, repl: string) -> string` : Replaces occurrences of `target` with `repl`.
- `repeat(s: string, count: int) -> string` : Repeats string `count` times.
- `padstart(s: string, targetlen: int, pad: string) -> string / padStart(...)` : Prepends `pad` until `targetlen`.
- `padend(s: string, targetlen: int, pad: string) -> string / padEnd(...)` : Appends `pad` until `targetlen`.
- `in() -> string / input() -> string` : Reads a line from standard input.
- `env(key: string, fallback: string) -> string` : Reads environment variable `key` or returns `fallback`.
- `debug(s: string) -> string` : Returns escaped, quoted debug representation of string.

```
let greeting = "  salivo world  ";
let clean = trim(greeting);
let up = upper(clean);
let has_world = contains(up, "WORLD");
outln($"Result: {up}, Found: {has_world}");
```

5.2 `salivo.std.strings`

Advanced UTF-8 string manipulation, owned `String`, and zero-copy `StrSlice` views.

Types

- `String`: Owned growable heap string (`ptr`, `len`, `cap`).
- `StrSlice`: Borrowed zero-copy string slice view (`ptr`, `len`).
- `ByteSlice`: Raw unvalidated byte buffer view (`ptr`, `len`).
- `CharIterator`: Unicode scalar code point iterator (`ptr`, `len`, `index`).

Functions

- `stringNew() -> String` / `stringnew()`: Creates new empty owned string.
- `stringWithCap(cap: int) -> String` / `stringwithcap()`: Preallocates string with capacity `cap`.
- `fromStr(s: StrSlice) -> String` / `fromstr()`: Allocates owned `String` from `StrSlice`.
- `asStr(s: String) -> StrSlice` / `asstr()`: Borrows `StrSlice` from owned `String`.
- `tryFromParts(ptr: int, len: int) -> Option<StrSlice>` / `tryfromparts()`: Validates that memory region contains valid UTF-8 and returns `Some(StrSlice)` or `None`.
- `fromUtf8(bytes: ByteSlice) -> Option<StrSlice>` / `fromutf8()`: Validates a raw `ByteSlice` and returns borrowed `StrSlice` if valid UTF-8.
- `fromparts(ptr: int, len: int) -> StrSlice` / `fromRawParts()`: Low-level unchecked raw constructor (explicit FFI/runtime escape hatch; does not validate UTF-8).
- `fromCString(s: string) -> String` / `fromcstring()`: Creates owned `String` from string literal.
- `toCString(s: String) -> string` / `tocstring()`: Converts `String` to C-string.
- `cStringView(s: string) -> StrSlice` / `cstringview()`: Creates borrowed `StrSlice` from literal.
- `subsliceStr(s: StrSlice, start: int, end: int) -> StrSlice`: Zero-copy subslice.
- `concat(a: String, b: String) -> String`: Concatenates two owned strings.
- `pushStr(s: String, appendtext: StrSlice) -> String` / `pushstr()`: Appends slice to string.
- `push(s: String, codepoint: int) -> String`: Appends Unicode codepoint.
- `pushByte(s: String, b: int) -> String` / `pushbyte()`: Appends raw byte.
- `clear(s: String) -> String`: Truncates string to length 0.
- `reserve(s: String, mincap: int) -> String`: Ensures string capacity.
- `chars(s: String) -> CharIterator`: Initializes codepoint iterator.

- `nextchar(it: CharIterator) -> Option<int>`: Reads current codepoint.
- `stepchar(it: CharIterator) -> CharIterator`: Advances iterator by 1 codepoint.
- `unicodeToUpper(cp: int) -> int` / `unicodetoupper(cp: int) -> int`: Unicode-aware 1:1 scalar uppercase mapping (covering ASCII, Latin-1, Latin Extended-A, Greek, Cyrillic, and Fullwidth forms).
- `unicodeToLower(cp: int) -> int` / `unicodetolower(cp: int) -> int`: Unicode-aware 1:1 scalar lowercase mapping (covering ASCII, Latin-1, Latin Extended-A, Greek, Cyrillic, and Fullwidth forms).
- `toUpper(s: String) -> String` / `upper(s: String) -> String`: Converts String to uppercase using Unicode-aware 1:1 scalar case conversion.
- `toLowerCase(s: String) -> String` / `lower(s: String) -> String`: Converts String to lowercase using Unicode-aware 1:1 scalar case conversion.
- `rfind(s: String, sub: String) -> int`: Finds last index of `sub`.
- `assertStringEq(a: String, b: String, msg: string)` / `assertstringeq(...)`: Asserts string equality.
- `assertSliceEq(a: StrSlice, b: StrSlice, msg: string)` / `assertsliceeq(...)`: Asserts slice equality.

NOTE

Unicode Case Mapping Architecture & Structural Scope:

`unicodeToUpper`, `unicodeToLower`, `toUpper`, and `toLowerCase` provide high-performance **Unicode-aware 1:1 scalar code point case mapping** across major global scripts (ASCII, Latin-1 Supplement, Latin Extended-A, Greek/Coptic, Cyrillic, and Fullwidth Latin).

Because scalar functions map a single code point to a single code point (`int -> int`), 1:1 scalar conversion preserves code point length invariants. Multi-code point expansions (such as German `ß` expanding to `ss`, or ligatures such as `fi` expanding to `fi`) and contextual/locale-specific casing (such as Turkish dotted/dotless `İ` / `ı`) are outside the 1:1 scalar mapping model. Unmapped code points outside the curated set are safely preserved without modification or corruption.

TIP

Authoritative Length & Embedded NUL (`\0`) Preservation:

Both `String` (`{ ptr: int, len: int, cap: int }`) and `StrSlice` (`{ ptr: int, len: int }`) treat their explicit `len` field as authoritative across all compiler lowering passes and standard library operations. Unlike legacy C strings, strings in Salvo fully support arbitrary binary byte sequences and embedded NUL bytes (`\0`) without premature `strlen()` truncation.

 SALVO

```
>>salvo.std.strings;

let mut s = stringNew();
s = pushStr(s, cStringView("Salvo"));
s = push(s, 32); // Space
s = pushStr(s, cStringView("Language"));

let slice_view = asStr(s);
assertEq(slice_view.len, 15, "Slice length mismatch");
```

5.3 `salvo.std.collections`

High-performance generic data structures.

1. Vector (`Vec<T>`)

- `vecNew<T>() -> Vec<T>` / `vecnew<T>()` : Creates an empty dynamic vector.
- `vecCap<T>(cap: int) -> Vec<T>` / `veccap<T>()` : Creates a vector with initial capacity.
- `push<T>(v: Vec<T>, val: T) -> Vec<T>` : Appends an element with exponential growth.
- `pop<T>(v: Vec<T>) -> Option<T>` : Removes and returns the last element.
- `get<T>(v: Vec<T>, index: int) -> T` : Reads element at `index` with bounds checking.
- `set<T>(v: Vec<T>, index: int, val: T)` : In-place update of element at `index`.
- `swapRemove<T>(v: Vec<T>, index: int) -> Vec<T>` / `swap_remove<T>()` : $O(1)$ removal by replacing target with last element.
- `insert<T>(v: Vec<T>, index: int, val: T) -> Vec<T>` : Inserts element at index.
- `remove<T>(v: Vec<T>, index: int) -> Vec<T>` : Removes element at index with element shifting.
- `vecLen<T>(v: Vec<T>) -> int` : Returns current element count.
- `capacity<T>(v: Vec<T>) -> int` : Returns allocated capacity.
- `empty<T>(v: Vec<T>) -> bool` / `isEmpty<T>()` : Checks if vector is empty.
- `reserve<T>(v: Vec<T>, additional: int) -> Vec<T>` : Reserves additional capacity.
- `reserveExact<T>(v: Vec<T>, additional: int) -> Vec<T>` / `reserve_exact<T>()` : Reserves exact capacity.
- `truncate<T>(v: Vec<T>, newLen: int) -> Vec<T>` : Truncates to `newLen`, dropping discarded elements.
- `resize<T>(v: Vec<T>, newLen: int, defaultVal: T) -> Vec<T>` : Resizes with default.
- `clear<T>(v: Vec<T>) -> Vec<T>` : Clears all elements, retaining capacity.
- `vecIter<T>(v: Vec<T>) -> VecIterator<T>` : Creates forward cursor iterator.
- `vecNext<T>(it: VecIterator<T>) -> Option<T>` : Advances iterator.
- `vecEq<T>(a: Vec<T>, b: Vec<T>) -> bool` : Element-wise vector equality.
- `vecDebug<T>(v: Vec<T>) -> string` : Debug string representation `[elem1, elem2]`.

2. Double-Ended Queue (`Deque<T>` / `VecDeque<T>`)

- `dequeNew<T>() -> Deque<T>` / `deqNew<T>()` : Creates a ring-buffer double-ended queue.
- `pushFront<T>(dq: Deque<T>, val: T) -> Deque<T>` : Prepends element at front in amortized $O(1)$.
- `pushBack<T>(dq: Deque<T>, val: T) -> Deque<T>` : Appends element at back in amortized $O(1)$.
- `popFront<T>(dq: Deque<T>) -> Option<T>` : Removes and returns front element.
- `popBack<T>(dq: Deque<T>) -> Option<T>` : Removes and returns back element.
- `deqget<T>(dq: Deque<T>, index: int) -> T` : Reads element at logical index `index`.
- `deqlen<T>(dq: Deque<T>) -> int` : Returns current number of elements.
- `deqIsEmpty<T>(dq: Deque<T>) -> bool` : Returns `true` if deque is empty.
- `deqIter<T>(dq: Deque<T>) -> DequeIterator<T>` : Creates forward iterator.
- `deqNext<T>(it: DequeIterator<T>) -> Option<T>` : Advances deque iterator.
- `dequeEq<T>(a: Deque<T>, b: Deque<T>) -> bool` : Deque equality.
- `dequeDebug<T>(dq: Deque<T>) -> string` : Debug representation `[elem1, elem2]`.

3. Hash Map (`HashMap<K, V>`)

- `mapNew<K, V>() -> HashMap<K, V>` : Creates an empty open-addressing hash table with linear probing.
- `put<K, V>(map: HashMap<K, V>, key: K, val: V) -> HashMap<K, V>` / `insert<K, V>()` : Inserts or updates key, automatically rehashing at 75% load factor.
- `lookup<K, V>(map: HashMap<K, V>, key: K) -> Option<V>` : Retrieves value for key without truncation on embedded NUL bytes.
- `has<K, V>(map: HashMap<K, V>, key: K) -> bool` / `containsKey<K, V>()` : Checks key existence.
- `delete<K, V>(map: HashMap<K, V>, key: K) -> HashMap<K, V>` / `remove<K, V>()` : Removes key, marking slot as tombstone for reuse.
- `mapLen<K, V>(map: HashMap<K, V>) -> int` : Returns count of active entries.
- `mapIsEmpty<K, V>(map: HashMap<K, V>) -> bool` : Returns `true` if map is empty.
- `mapIter<K, V>(map: HashMap<K, V>) -> MapIterator<K, V>` : Creates bucket cursor iterator.
- `mapNext<K, V>(it: MapIterator<K, V>) -> Option<MapEntry<K, V>>` : Advances map iterator.
- `mapEq<K, V>(a: HashMap<K, V>, b: HashMap<K, V>) -> bool` : Key-value map equality.
- `mapDebug<K, V>(map: HashMap<K, V>) -> string` : Debug representation `{k1: v1, k2: v2}`.

4. Hash Set (`HashSet<T>`)

- `setNew<T>() -> HashSet<T>` : Creates an empty hash set built on `HashMap<T, int>`.
- `add<T>(set: HashSet<T>, val: T) -> HashSet<T>` : Adds element to set.
- `discard<T>(set: HashSet<T>, val: T) -> HashSet<T>` : Removes element from set.
- `member<T>(set: HashSet<T>, val: T) -> bool` / `setContains<T>()` : Checks element membership.
- `setLen<T>(set: HashSet<T>) -> int` : Returns element count.
- `setIsEmpty<T>(set: HashSet<T>) -> bool` : Checks if set is empty.
- `setUnion<T>(a: HashSet<T>, b: HashSet<T>) -> HashSet<T>` : Computes union $A \cup B$.
- `setIntersection<T>(a: HashSet<T>, b: HashSet<T>) -> HashSet<T>` : Computes intersection $A \cap B$.
- `setDifference<T>(a: HashSet<T>, b: HashSet<T>) -> HashSet<T>` : Computes difference $A \setminus B$.
- `setIter<T>(set: HashSet<T>) -> SetIterator<T>` : Creates set iterator.
- `setNext<T>(it: SetIterator<T>) -> Option<T>` : Advances set iterator.
- `setEq<T>(a: HashSet<T>, b: HashSet<T>) -> bool` : Set equality.
- `setDebug<T>(set: HashSet<T>) -> string` : Debug representation `{elem1, elem2}`.

5. Stack (`Stack<T>`) & Queue (`Queue<T>`) Adapters

- `stackNew<T>() -> Stack<T>` : Creates a LIFO stack adapter over `Vec<T>`.
- `stackPush<T>(s: Stack<T>, val: T) -> Stack<T>` : Pushes element onto stack.
- `stackPop<T>(s: Stack<T>) -> Option<T>` : Pops element from stack.
- `stackPeek<T>(s: Stack<T>) -> Option<T>` : Peeks at top element.

- `stackLen<T>(s: Stack<T>) -> int`: Returns stack depth.
- `stackIsEmpty<T>(s: Stack<T>) -> bool`: Checks if stack is empty.
- `queueNew<T>() -> Queue<T>`: Creates a FIFO queue adapter over ring-buffer `Deque<T>`.
- `enqueue<T>(q: Queue<T>, val: T) -> Queue<T>`: Enqueues element at tail.
- `dequeue<T>(q: Queue<T>) -> Option<T>`: Dequeues element from head.
- `queuePeek<T>(q: Queue<T>) -> Option<T>`: Peeks at head element.
- `queueLen<T>(q: Queue<T>) -> int`: Returns queue length.
- `queueIsEmpty<T>(q: Queue<T>) -> bool`: Checks if queue is empty.

```

SALIVO

><salivo.std.collections::{Vec, vecNew, push, get, vecLen, HashMap, mapNew, put, lookup, Deque, dequeNew, pu

// Vector Example
let mut v = vecNew<int>();
v = push<int>(v, 100);
v = push<int>(v, 200);
let val = get<int>(v, 0); // 100

// Hash Map Example
let mut users = mapNew<int, string>();
users = put<int, string>(users, 101, "User A");
users = put<int, string>(users, 102, "User B");
let user_opt = lookup<int, string>(users, 101);
if (user_opt.is_some) {
    outln($"Found User: {user_opt.value}");
}

```

5.4 `salivo.std.math`

Constants, trigonometry, logarithms, arithmetic invariants, and random numbers.

Constants

`PI` (3.14159...), `TAU` (6.28318...), `E` (2.71828...), `LN2`, `LN10`, `SQRT2`, `EPSILON`, `NAN_VAL`, `INFINITY_VAL`, `NEG_INFINITY_VAL`.

Functions

- `abs(v: int) -> int` / `absfloat(v: float) -> float`: Absolute value.
- `min(a: int, b: int) -> int` / `max(a: int, b: int) -> int`: Minimum and maximum.
- `clamp(val: int, min_val: int, max_val: int) -> int`: Clamps value in range.
- `pow(base_val: int, exp_val: int) -> int`: Integer power.
- `sqr(v: float) -> float`: Square root (Newton-Raphson).
- `cbrt(v: float) -> float`: Cube root.
- `sin(rad: float) -> float` / `cos(rad: float) -> float` / `tan(rad: float) -> float`: Trigonometric functions.

- `asin(v: float) -> float` / `acos(v: float) -> float` / `atan(v: float) -> float` / `atan2(y: float, x: float) -> float`: Inverse trigonometry.
- `exp(v: float) -> float`: Exponential function e^v .
- `log(v: float) -> float` / `log2(v: float) -> float` / `log10(v: float) -> float`: Logarithms.
- `floor(v: float) -> int` / `ceil(v: float) -> int` / `round(v: float) -> int` / `trunc(v: float) -> int`: Rounding.
- `fract(v: float) -> float`: Fractional part.
- `satadd(a: int, b: int, max_val: int) -> int` / `satsub(a: int, b: int, min_val: int) -> int`: Saturating arithmetic.
- `checkedadd(a: int, b: int, max_val: int) -> Option<int>`: Overflow-checked arithmetic.
- `wrapadd(a: int, b: int, max_val: int) -> int`: Wrapping arithmetic.
- `seed(s: int) -> Seed`: Initializes PRNG seed.
- `rng(s: Seed) -> RandomGenerator`: Initializes PRNG state.
- `nextu32(rng: RandomGenerator) -> int`: Generates next random 32-bit integer.
- `nextf32(rng: RandomGenerator) -> float`: Generates float in $[0.0, 1.0]$.
- `range(rng: RandomGenerator, min_val: int, max_val: int) -> int`: Random integer in $[min, max]$.

```

SALIVO

><salivo.std.math;

let s = seed(12345);
let r = rng(s);
let die_roll = range(r, 1, 6);
let hyp = sqrt(3.0 * 3.0 + 4.0 * 4.0); // 5.0

```

5.5 `salivo.std.io`

File streams, standard I/O streams, binary buffering, and directories.

Types

- `File`: File stream handle (`handle_id`, `position`, `is_open`, `arc_ref`).
- `Directory`: Directory handle (`handle_id`, `path`).
- `Metadata`: File size, directory flag, readonly status.
- `IoError` & `IoErrorKind` (`NotFound`, `PermissionDenied`, `AlreadyExists`, `InvalidInput`, etc.).

Functions

- `open(path: string) -> Result<File, IoError>`: Opens existing file.
- `create(path: string) -> Result<File, IoError>`: Creates/truncates file.
- `isopen(f: File) -> bool`: Checks open status.
- `close(f: File) -> Result<bool, IoError>`: Closes file stream.

- `read(f: File, buf_ptr: int, count_bytes: int) -> Result<int, IOError>` : Reads bytes into buffer.
- `write(f: File, buf_ptr: int, count_bytes: int) -> Result<int, IOError>` : Writes bytes from buffer.
- `flush(f: File) -> Result<bool, IOError>` : Flushes file buffer.
- `seek(f: File, offset: int, whence: int) -> Result<int, IOError>` : Seeks position.
- `tell(f: File) -> Result<int, IOError>` : Gets current position.
- `delete(path: string) -> Result<bool, IOError>` : Removes file.
- `exists(path: string) -> bool` : Checks file existence.
- `mkdir(path: string) -> Result<Directory, IOError>` : Creates directory.
- `rmdir(path: string) -> Result<bool, IOError>` : Removes directory.
- `dirlist(path: string) -> Result<string, IOError>` : Lists entries.
- `dircount(path: string) -> Result<int, IOError>` : Entry count.
- `pwd() -> string` : Returns current working directory.
- `cd(path: string) -> Result<bool, IOError>` : Changes directory.
- `readstr(path: string) -> Result<string, IOError>` : Reads full file to string.
- `writestr(path: string, content: string) -> Result<bool, IOError>` : Writes string to file.
- `appendstr(path: string, content: string) -> Result<bool, IOError>` : Appends string to file.
- `readline() -> Result<string, IOError>` / `inln() -> string` : Reads stdin line.
- `inint() -> int` / `infloat() -> float` : Reads typed values from stdin.
- `readbytes(f: File, count_bytes: int) -> Result<int, IOError>` : Allocates and reads bytes.
- `writebytes(f: File, buffer_addr: int, count_bytes: int) -> Result<int, IOError>` : Writes raw bytes.

```

SALIVO

><salivo.std.io;

let write_res = writestr("config.json", "{\"status\": \"active\"}");
if (write_res.is_ok) {
    let content_res = readstr("config.json");
    if (content_res.is_ok) {
        outln($"Read: {content_res.value}");
    }
}
}

```

5.6 `salivo.std.fs`

Path normalization, file metadata inspection, and low-level OS file descriptors.

Types & Functions

- `path(p: string) -> Path` : Constructs Path wrapper.
- `normalize(path: string) -> string` : Converts backslashes to forward slashes.
- `pathJoin(path1: string, path2: string) -> string` / `pathjoin(...)` : Joins two paths safely.

- `parent(path: string) -> string`: Returns parent directory path.
- `filename(path: string) -> string`: Returns base filename.
- `ext(path: string) -> string`: Returns file extension without dot.
- `absolute(path: string) -> string`: Resolves absolute path.
- `relative(base: string, target: string) -> string`: Computes relative path.
- `pathExists(path: string) -> bool` / `pathexists(...)`: Checks file or directory existence.
- `open(path: string, mode: string) -> File`: Opens file with mode (`"r"`, `"w"`, `"a"`, `"rb+"`).
- `close(file: File)` / `flush(file: File)` / `sync(file: File)`
- `meta(file: File) -> FileMetadata`: Size, timestamps, permissions.
- `read(file: File, buffer: int, maxbytes: int) -> int`: Reads up to `maxbytes`.
- `readByte(file: File) -> int` / `readbyte(...)`: Reads single byte.
- `seek(file: File, offset: int) -> int` / `tell(file: File) -> int` / `eof(file: File) -> bool`
- `write(file: File, val: int) -> int`: Writes single byte.
- `append(file: File, val: int) -> int`: Appends byte at end of file.
- `mkdir(path: string) -> Directory` / `rmdir(path: string)` / `dirExists(path: string) -> bool` / `dirList(path: string) -> string`

```

SALIVO

><salivo.std.fs::{pathJoin, ext, filename, parent};

let full_path = pathJoin("C:/salivo/projects", "server.sal");
let file_ext = ext(full_path);           // "sal"
let file_base = filename(full_path);     // "server.sal"
let parent_dir = parent(full_path);      // "C:/salivo/projects"

```

5.7 `salivo.std.crypto`

Cryptographic hashing, symmetric encryption, and OS entropy generation.

Types

- `Sha256`, `Sha512`, `Blake3`: Hash contexts.
- `Aes128`, `Aes256`: Symmetric ciphers.
- `EntropyProvider`, `RandomDevice`, `SecureRandom`.
- `CryptoError` & `CryptoErrorKind`.

Functions

- `sha256() -> Sha256` / `sha512() -> Sha512` / `blake3() -> Blake3`: Creates hash engines.
- `sha256Hex(input_raw: string) -> string` / `sha256hex(...)`: Computes 64-character SHA-256 hex digest.
- `aes128(key_addr: int) -> Result<Aes128, CryptoError>`: Creates AES-128 cipher.

- `aes256(key_addr: int) -> Result<Aes256, CryptoError>`: Creates AES-256 cipher.
- `entropy(avail: bool, source_id: int) -> EntropyProvider`: Initializes entropy source.
- `secSeed(provider: EntropyProvider) -> Result<int, CryptoError>` / `secseed(...)`: Secure 64-bit seed from OS CSPRNG.
- `randBytes(count_bytes: int, provider: EntropyProvider) -> Result<int, CryptoError>` / `randbytes(...)`: Secure random bytes.
- `encrypt(cipher: Aes256, plaintext_addr: int, plaintext_len: int) -> Result<int, CryptoError>`: Encrypts memory buffer.
- `decrypt(cipher: Aes256, ciphertext_addr: int, ciphertext_len: int) -> Result<int, CryptoError>`: Decrypts memory buffer.

```

SALIVO

><salivo.std.crypto::{sha256Hex};

let hash = sha256Hex("salivo-production-payload");
outln($"SHA-256: {hash}");

```

5.8 `salivo.std.sync`

Thread synchronization primitives, mutual exclusion, condition variables, channels, and atomic memory operations.

Types & Functions

- **Mutex:**
 - `mutex() -> Mutex`: Creates new OS mutex.
 - `lock(m: Mutex) -> MutexGuard`: Acquires mutual exclusion.
 - `trylock(m: Mutex) -> bool`: Non-blocking acquisition.
 - `unlock(g: MutexGuard)`: Releases lock.
 - `islocked(m: Mutex) -> bool` / `owner(m: Mutex) -> int`
- **RwLock (Read-Write Lock):**
 - `rwlock() -> RwLock`
 - `readlock(rw: RwLock) -> ReadGuard` / `tryread(rw: RwLock) -> bool` / `unlockread(g: ReadGuard)`
 - `writelock(rw: RwLock) -> WriteGuard` / `trywrite(rw: RwLock) -> bool` / `unlockwrite(g: WriteGuard)`
 - `readcount(rw: RwLock) -> int` / `iswritelocked(rw: RwLock) -> bool`
- **Condvar (Condition Variable):**
 - `condvar() -> Condvar`
 - `wait(cv: Condvar, m: Mutex)`: Waits for condition notification.
 - `notify(cv: Condvar) -> bool`: Wakes one waiting worker.
 - `signal(cv: Condvar) -> bool`: Wakes all waiting workers.
 - `waitercount(cv: Condvar) -> int`: Active wait count.

- **Channel:**

- `channel() -> Channel`: Creates MPMC message channel.
- `sender(ch: Channel) -> Sender` / `receiver(ch: Channel) -> Receiver`
- `send(s: Sender, value: int) -> bool`: Sends integer value.
- `recv(r: Receiver) -> int`: Blocking receive.
- `tryrecv(r: Receiver) -> Option<int>`: Non-blocking receive.
- `close(ch: Channel)` / `isclosed(ch: Channel) -> bool` / `size(ch: Channel) -> int`

- **Atomics:**

- `atomic(initial: int) -> AtomicInt`
- `load(a: AtomicInt) -> int` / `store(a: AtomicInt, value: int)`
- `swap(a: AtomicInt, value: int) -> int`
- `cas(a: AtomicInt, expected: int, desired: int) -> bool`: Compare-and-swap.
- `fetchadd(a: AtomicInt, value: int) -> int` / `fetchsub(a: AtomicInt, value: int) -> int`
- `atomicbool(initial: bool) -> AtomicBool` / `loadbool` / `storebool` / `swapbool`

```

SALIVO

><salivo.std.sync;

let m = mutex();
let guard = lock(m);
// Critical section operations...
unlock(guard);

let counter = atomic(0);
fetchadd(counter, 1);
let current = load(counter); // 1

```

5.9 `salivo.std.time`

High-precision monotonic clock, timestamps, ISO-8601 formatting, and timers.

Types

- `Duration` (`seconds`, `nanoseconds`).
- `Instant` (`ticks_nanos`).
- `Date` (`year`, `month`, `day`), `Time` (`hour`, `minute`, `second`, `nanosecond`), `DateTime`, `Timezone`.
- `Timer`, `Stopwatch`.

Functions

- `duration(sec: int, nanos: int) -> Duration`: Constructs duration.
- `fromms(ms: int) -> Duration`: Converts milliseconds to duration.
- `now() -> Instant`: Returns monotonic timestamp.

- `epoch() -> int`: Returns Unix timestamp in milliseconds.
- `elapsed(start_t: Instant) -> Duration`: Computes duration since `start_t`.
- `sleepdur(dur: Duration)` / `sleep(ms: int)`: Sleeps for duration.
- `sleepuntil(target_time_ms: int)`: Sleeps until target epoch time.
- `timer(delay_ms: int) -> Timer`: Creates timer.
- `isactive(t: Timer) -> bool` / `canceltimer(t: Timer) -> bool` / `processtimers() -> int`
- `addduration(d1: Duration, d2: Duration) -> Duration`
- `subduration(d1: Duration, d2: Duration) -> Duration`
- `diffinstant(i1: Instant, i2: Instant) -> Duration`
- `format(dt: DateTime, fmt_str: string) -> string`: Formats to ISO-8601 string.
- `parse(date_str: string, fmt_str: string) -> Result<DateTime, string>`: Parses date string.

```

SALIVO

><salivo.std.time;

let start = now();
sleep(50);
let dur = elapsed(start);
outln($"Elapsed: {dur.nanoseconds} ns");

```

5.10 `salivo.std.net`

Operating system IPv4/IPv6 networking, TCP streams, listeners, UDP, and DNS resolution.

Types

- `IPAddress` (`version`, `octet1`, `octet2`, `octet3`, `octet4`).
- `SocketAddress` (`ip`, `port`).
- `TcpStream`, `TcpListener`, `UdpSocket`, `DnsResolver`.
- `NetworkError` & `NetworkErrorKind`.

Functions

- `ipv4(a: int, b: int, c: int, d: int) -> IPAddress` / `ipv6(...)`
- `sockaddr(ip: IPAddress, port: int) -> SocketAddress`
- `parseip(ip_str: string) -> IPAddress`: Parses `"192.168.1.1"`.
- `resolve(domain_name: string, provider: PlatformProvider) -> Result<IPAddress, NetworkError>`: DNS lookup.
- `tcpsendbuf(stream: TcpStream, buf_ptr: int, count_bytes: int) -> Result<int, NetworkError>`
- `tcpsendstr(stream: TcpStream, data: string) -> Result<int, NetworkError>`
- `tcprecvstr(stream: TcpStream, max_bytes: int) -> Result<string, NetworkError>`

- `shutdown(stream: TcpStream) -> Result<bool, NetworkError>`
- Low-level socket intrinsics: `socket()`, `connect()`, `bind()`, `listen()`, `accept()`, `sockread()`, `sockwrite()`, `close()`.

```

SALIVO

><salivo.std.net;

let ip = parseip("127.0.0.1");
let addr = sockaddr(ip, 8080);

```

5.11 `salivo.std.task`

Cooperative concurrency, tasks, structured concurrency scopes, and cancellation.

Types & Functions

- `Task` (`state_ptr`, `poll_fn`, `TaskScope`, `CancellationToken`).
- `Poll<T>`: Variants `Poll::Ready(T)` or `Poll::Pending`.
- `TaskState`: `Created`, `Ready`, `Running`, `Pending`, `Completed`, `Destroyed`.
- `spawntask(root_task: Task) -> int`: Spawns async task.
- `spawnval(val: int) -> int`: Spawns value task.
- `fail(task_id: int, msg: string)` / `isfailed(task_id: int) -> bool`
- `wake(task_id: int)`: Wakes suspended task.
- `polltask(t: Task) -> Poll<int>`: Polls task for completion.
- `gettaskstate(task_id: int) -> TaskState` / `settaskstate(task_id: int, state: TaskState)`
- `blockon(root_task: Task) -> int`: Blocks current thread on root task.
- `scope() -> TaskScope`: Creates structured concurrency scope.
- `scopespawn(sc: TaskScope, task: Task) -> int`: Spawns task into scope.
- `scopejoin(sc: TaskScope, task_id: int) -> int`: Joins specific child.
- `scopecancel(sc: TaskScope)`: Cancels all tasks in scope.
- `joinall(sc: TaskScope) -> int` / `waitany(sc: TaskScope) -> int` / `waitall(sc: TaskScope) -> int`
- `token() -> CancellationToken` / `canceltoken(tok: CancellationToken)` / `istokenactive(tok: CancellationToken) -> bool`

```

SALIVO

><salivo.std.task;

let sc = scope();
let t = Task { state_ptr: 42, poll_fn: 0 };
let tid = scopespawn(sc, t);
let completed_val = scopejoin(sc, tid);

```

5.12 Auxiliary Modules

- `salivo.std.option` : `unwrap<T>` , `unwapor<T>` , `issome<T>` , `isnone<T>` .
- `salivo.std.result` : `unwrap<T, E>` , `unwaperr<T, E>` , `isok<T, E>` , `iserr<T, E>` .
- `salivo.std.process` : `exit(code: int)` , `cwd() -> string` .
- `salivo.std.random` : `rand() -> int` , `randrange(min_val: int, max_val: int) -> int` , `randfloat() -> float` .
- `salivo.std.hash` : `hashstr(s: string) -> int` (DJB2 hash), `hash(v: int) -> int` .
- `salivo.std.iter` : `rangeiter(start: int, end: int, step: int) -> RangeIterator` , `next(iter: RangeIterator) -> Option<int>` .

Part VI: Tooling, Compiler CLI & Project Ecosystem

The official Salivo CLI is `sf` (located at `C:\Users\sahil\.salivo\bin\sf.exe`).

6.1 Compiler Commands (`sf`)

```
BASH
# Run a Salivo script directly
sf run main.sal

# Compile to optimized native executable
sf build main.sal -o server.exe --release

# Maximum native optimization with host CPU vectorization
sf build main.sal -o server.exe --opt-level aggressive --native

# Type-check and validate without code generation
sf check main.sal

# Execute test suite functions in source files
sf test tests/

# Start interactive REPL
sf repl
```

6.2 Compiler Optimization Pipeline & Performance Tuning

The Salivo compiler infrastructure leverages an advanced Static Single Assignment (SSA) optimization engine feeding into LLVM16:

- **Optimization Levels** (`-O 0` to `-O 3`):
 - `-O 0` (Debug): Fast compilation, zero instruction reordering, preserves full debug line tables.
 - `-O 1` (Basic): Local peephole optimizations and dead value stripping.
 - `-O 2` (Standard Release): Global common subexpression elimination, function inlining, and standard loop simplifications.
 - `-O 3` / `--opt-level aggressive`: Aggressive loop unrolling, SLP auto-vectorization, memory-to-register promotion, and aggressive inlining.
- **Host CPU Vectorization** (`--native`): Emits code specifically tailored to the host instruction set architecture, leveraging AVX2, FMA, and BMI2 vector registers.
- **Zero-Cost SSA Optimization Passes:**

- **ArcOptimization** : Analyzes lifetime ranges and elides redundant paired ARC retain/release operations within basic blocks and loops, reducing atomic reference counting overhead to zero for local variables.
- **BoundsCheckElimination** : Employs dominator tree range analysis to verify index validity at loop entry, safely hoisting bounds checks out of inner iteration loops.
- **ConstantDivisorOptimization** : Evaluates constant non-zero divisors at compile time, eliminating branch conditions and panic splitting on arithmetic division and modulo operations.
- **DeadCodeElimination** (**DCE**) : Recursively strips dead branches, unreachable blocks, and unused temporary values prior to backend emission.

6.3 Package Manager (**spm**)

```
BASH

# Initialize a new Salivo project
spm init myproject

# Add an official package dependency
spm add @salivo/http

# Install dependencies defined in Salivo.toml
spm install

# Publish package to the Salivo registry
spm publish
```

6.4 Code Quality & Tooling

```
BASH

# Format code according to official style rules
salivofmt --check src/
salivofmt --write src/

# Run static analysis and linting
salivolint src/

# Generate API documentation
salivodoc src/ -o docs/
```

Part VII: Complete Working Examples

Example 1: High-Performance Data Pipeline with Collections & Hashing

```
● ● ● SALIVO

module app.pipeline;

><salivo.std.core::{assert, assertEq};
><salivo.std.collections::{vecNew, push, get, vecLen, mapNew, put, lookup};
><salivo.std.crypto::{sha256Hex};
><salivo.std.time::{now, elapsed};

func main() -> int {
    let start_time = now();
    outln("=== Starting Salivo Data Pipeline ===");

    // Initialize collections
    let mut data_vec = vecNew<string>();
    data_vec = push<string>(data_vec, "transaction_1001");
    data_vec = push<string>(data_vec, "transaction_1002");
    data_vec = push<string>(data_vec, "transaction_1003");

    let mut ledger = mapNew<string, string>();

    // Process and hash each item
    for (let mut i = 0; i < vecLen<string>(data_vec); i = i + 1) {
        let raw_tx = get<string>(data_vec, i);
        let digest = sha256Hex(raw_tx);
        ledger = put<string, string>(ledger, raw_tx, digest);
        outln($"[PROCESSED] {raw_tx:upper} -> Digest: {digest:slice(0, 16)}...");
    }

    // Verify lookup
    let query = lookup<string, string>(ledger, "transaction_1002");
    if (query.is_some) {
        outln($"Verified lookup for TX 1002: {query.value:slice(0, 8)}");
    }

    let dur = elapsed(start_time);
    outln($"Pipeline completed in {dur.nanoseconds} ns");
    return 0;
}
```

Example 2: Thread-Safe State & Atomic Concurrency

```
module app.concurrent_counter;

><salivo.std.core::{assertEq};
><salivo.std.sync::{mutex, lock, unlock, atomic, fetchadd, load};
><salivo.std.task::{scope, spawn_task, scopespawn, scopejoin, Task};

func main() -> int {
    outln("=== Salivo Concurrency & Sync Demonstration ===");

    let total_tasks = 10;
    let global_counter = atomic(0);
    let state_lock = mutex();

    let sc = scope();

    for (let mut i = 0; i < total_tasks; i = i + 1) {
        // Increment atomic counter safely
        fetchadd(global_counter, 1);

        let guard = lock(state_lock);
        outln($"Task {i:str} registered. Counter value: {load(global_counter):str}");
        unlock(guard);
    }

    let final_count = load(global_counter);
    assertEq(final_count, total_tasks, "Concurrency counter mismatch");
    outln($"All {final_count:str} operations completed successfully.");
    return 0;
}
```

Example 3: File System Inspector & Path Manipulation

```
module app.fs_inspector;

><salivo.std.fs::{pathJoin, normalize, filename, ext, pathExists, parent};
><salivo.std.io::{writestr, readstr, delete};

func main() -> int {
    let test_file = "salivo_test_artifact.tmp";
    let test_data = "Engine=Salivo\nVersion=1.0\nStatus=Operational";

    // Write file
    let write_status = writestr(test_file, test_data);
    if (!write_status.is_ok) {
        outln("Failed to create test artifact.");
        return 1;
    }

    // Path analysis
    let joined = pathJoin("workspace", test_file);
    outln($"Joined Path: {normalize(joined)}");
    outln($"Filename: {filename(test_file)}");
    outln($"Extension: {ext(test_file)}");

    // Read and verify
    let read_result = readstr(test_file);
    if (read_result.is_ok) {
        outln("File Contents:\n" + read_result.value);
    }

    // Cleanup
    delete(test_file);
    outln("Cleaned up test artifact.");
    return 0;
}
```